One ceiling, many filters: $c(x)$ across SPARK's value-based safe controllers

Expanded edition — builds on cx_derivation, gentle on calculus

SPARK goal-insertion notes

How to read this note

This is the sequel to `cx_derivation.pdf`. There we built, for the SSA filter, one number

$$c(x) \;=\; \sum_k u_{lim,k}\,\big|(L_g\phi)_k\big|$$

— the fastest the motors can drive the safety index ϕ down, given the joint speed limits — and showed SSA can keep the robot safe exactly when $c(x) \geq \eta$ (more generally $c(x) \geq \eta + L_f\phi$). If the symbols ∇, $\dot{\phi}$, $L_g\phi$, the dot product, or "minimize over the box" are fuzzy, read the Notation primer in `cx_derivation.pdf` first; we reuse all of it here without re-explaining.

The single message of this note: $c(x)$ does not change from one safe controller to another. It is a property of the robot and the contact, not of the algorithm. Every value-based controller in SPARK — SSA, SSS, CBF, their relaxed versions, p-SSA, and the heuristics SMA/PFM — measures the same $c(x)$ against a different demand, and reacts differently when the demand can't be met. Once you see that, all of them fall out of the SSA result with almost no new math.

The one picture: a fixed ruler, different demands

Think of $c(x)$ as a ruler that measures the robot's braking ability at the current pose: "the motors can pull ϕ down at most this fast." Each safe controller does two things:

1. it sets a demand d_i — the rate of decrease it insists on, $\dot{\phi}_i \leq -d_i$;
2. it has a reaction for when the ruler comes up short ($c < d$): give up, or bend the rule with slack, or just shove.

Everything below is: what is the demand, and what is the reaction. The feasibility algebra is identical to SSA every time.

The shared constraint row (why one analysis covers all)

Every value-based controller filters the reference control u_{ref} through the same shape of problem:

$$\min_{u\in\mathcal{U}} \ \tfrac{1}{2}\|u-u_{ref}\|_Q^2 \quad \text{s.t.} \quad \underbrace{L_f\phi_i + L_g\phi_i\, u + d_i}_{=\ \dot\phi_i + d_i} \ \le\ 0 \quad \text{for each active } i.$$

The objective ("stay close to what the task wanted") and the actuator box $\mathcal{U}$ are common to all. The kinematics inside the row — $L_g\phi_i = n_i^\top J_i(x)G(x)$, hence

$$c_i(x) = \max_{u\in\mathcal{U}} \big(-L_g\phi_i\, u\big) = \sum_k u_{lim,k}\,\big|(L_g\phi_i)_k\big|$$

— are identical for every controller, because they describe the body and the obstacle, not the policy. From `cx_derivation` Steps 2–4, the best decrease the box can produce for row i is $\dot\phi_i = L_f\phi_i - c_i(x)$, so the row is satisfiable iff

$$L_f\phi_i - c_i(x) \ \le\ -d_i \quad\Longleftrightarrow\quad \boxed{c_i(x) \ \ge\ d_i + L_f\phi_i}.$$

This is exactly the SSA inequality from Step 4 — with η replaced by the generic demand d_i. That single substitution is the whole generalization.

The universal margin $g(x)$

Bundle "can the box meet the demand?" into one signed number, taken over the active constraints:

$$\boxed{g(x) \ :=\ \min_{i \text{ active}} \ (c_i(x) - d_i - L_f\phi_i)}.$$

- $g(x) \ge 0$: the motors can hold $\dot\phi \le -d$ — the filter's guarantee stands.
- $g(x) < 0$: no control in the box can hold $\dot\phi \le -d$ — the guarantee is defeated, for every value-based controller at once, because g is built only from c (the box) and d (the demand).

For SSA with $L_f\phi = 0$ this is just $g = c - \eta$, the quantity you already know. What changes per controller is only d_i (and which i are active). The reaction to $g < 0$ is the only policy-specific part, and it is what distinguishes the families below.

The two demand models (this is the crux)

Almost every difference reduces to how the demand depends on how deep you are in danger ($\phi > 0$ = in danger, deeper = larger ϕ).

Constant demand $d = \eta$ (SSA, r-SSA, p-SSA). "Whenever you are in danger at all, retreat at rate at least η." This is blunt: even a hair past the boundary ($\phi = 0^+$) it insists on the full η. So a single low-authority contact with $c(x) < \eta$ defeats it right at the boundary. Easy to trigger.

Proportional demand $d = \lambda\phi$ (SSS, CBF, and their relaxed forms). "Retreat faster the deeper you are." At the boundary $\phi = 0$ the demand is $\lambda \cdot 0 = 0$, so any $c \ge 0$ satisfies it — grazing contact

cannot defeat it. You only win once you have penetrated far enough that the demand overtakes the ruler:

$$\lambda\phi \;>\; c(x) \quad\Longleftrightarrow\quad \phi \;>\; c(x)/\lambda.$$

This is the classic barrier idea: $\dot{\phi} \le -\lambda\phi$ forces ϕ to decay like $e^{-\lambda t}$, asking almost nothing near the surface and ramping up inside. Same ruler $c(x)$, gentler demand near the boundary $\Rightarrow$ a much smaller attack surface. (Empirically, with $\eta = 0.5,\ \lambda = 10$ over 21 seeds: the constant-η family had 6–8 vulnerable seeds, the $\lambda\phi$ family only 2 — exactly because the $\lambda\phi$ demand vanishes at the boundary, so the attacker needs a reachable deeply-penetrating contact, which is rarer.)

Family A — hard QP filters: SSA, SSS, CBF

No slack variable. When $g < 0$ the constraint polytope is empty, the QP is primal-infeasible, and the controller gives up and returns u_{ref} (the raw, possibly-colliding command). $c(x)$ test applies verbatim; only d_i and the active set differ:

- SSA — $d_i = \eta$, active when $\phi_i \ge 0$. Defeated at the boundary if $c < \eta$.
- SSS (Sublevel Safe Set) — $d_i = \lambda\phi_i$, active when $\phi_i \ge 0$. Same rule as SSA but with the proportional demand: softer at the boundary, defeated only for $\phi > c/\lambda$.
- CBF (Control Barrier Function) — $d_i = \lambda\phi_i$, but active for every masked constraint, even those with $\phi_i < 0$ (there the demand $\lambda\phi_i < 0$ is trivially met). The extra always-on rows make it preventive — it brakes you before you reach the boundary — and give it more rows to form the multi-constraint pincer of `cx_derivation` Step 9. The binding (defeating) case is still $\phi > 0$ with $\lambda\phi > c$.

Family B — soft QP filters: r-SSA, r-CBF, r-SSS, p-SSA

These add a slack $s_i \ge 0$ to each row, $\dot{\phi}_i \le -d_i + s_i$, and penalize it in the objective. Because s_i can be as large as needed, the QP is always feasible — the filter never "gives up." So how can $c(x)$ still defeat them?

It governs the forced size of the slack. When $g < 0$ (the would-be-infeasible region), even the best box control leaves $\dot{\phi} = L_f\phi - c$, so the slack must be at least

$$s_i^\star \;\ge\; (d_i + L_f\phi_i - c_i)_+ \;=\; |g|_+.$$

A nonzero s_i is a guaranteed violation of the safety rate — the filter is no longer holding $\dot{\phi} \le -d$. What that violation looks like depends on the slack penalty weight ρ:

- low ρ: cheap to violate, so the controller pushes through the obstacle — collision;
- high ρ: expensive to violate, so it freezes rather than pay — stall / deadlock ($u \to 0$).

Crucially, the set of defeated states is identical to the hard version (same $g < 0$); soft vs. hard changes only the symptom, not the vulnerable set. (This is why, in the sweep, the vulnerable seeds matched within each demand family regardless of slack.)

p-SSA deserves a special line. Its Phase I minimizes $\|s\|^2$ — the smallest slack that restores feasibility — which for a single active constraint is exactly $(d + L_f\phi - c)_+ = |g|_+$. So p-SSA's projection magnitude equals the $c(x)$ margin; there is nothing to estimate. Phase II then tracks u_{ref} subject to the just-relaxed rows.

Family C — heuristics: SMA, PFM ($c(x)$ as a bound, not an equation)

These solve no QP, so there is no feasible/infeasible or slack dichotomy — but $c(x)$ still bounds them.

- SMA (Sliding Mode) sets $u = u_{ref} - c_{sma}\, L_g\phi_{\max}$ (a fixed step opposite the worst constraint's gradient) and then clips to the box. Any $u \in \mathcal{U}$ obeys $\dot{\phi}_i \geq L_f\phi_i - c_i(x)$, so $c_i(x)$ is the post-clip floor: if $L_f\phi_i \geq c_i(x)$, no admissible command can reduce ϕ_i at all. Thus $g < 0$ is necessary for SMA to fail — but because it takes a fixed step instead of seeking the optimal box corner, it fails in a superset of the QP-infeasible region. $c(x)$ is a one-sided bound, not the exact threshold.
- PFM (Potential Field) acts in Cartesian space and maps back with a pseudo-inverse without clipping to $\mathcal{U}$, so it can command physically-unrealizable controls. $c(x)$ describes what is actually reachable, but PFM may ignore it, so the correspondence is only qualitative.

Verdict table

controller	demand d_i	reaction to $g < 0$	$c(x)$ certificate
SSA	η (const)	give up $\to u_{ref}$	exact: $g = c - \eta$
SSS	$\lambda\phi$	give up $\to u_{ref}$	exact; needs $\phi > c/\lambda$
CBF	$\lambda\phi$ (all rows)	give up $\to u_{ref}$	exact; preventive
r-SSA / r-CBF / r-SSS	η or $\lambda\phi$	forced slack $\geq \|g\|$	exact (slack floor)
p-SSA	η	projection $= \|g\|_+$	exact (equals margin)
SMA	—	clipped fixed step	necessary bound only
PFM	—	ignores the box	loose

In one sentence. $c(x) = \sum_k u_{lim,k}|(L_g\phi)_k|$ is a fixed ruler — the motors' maximum braking. Each controller picks a demand (constant η or depth-proportional $\lambda\phi$) and a reaction (give up / slack / project / shove), but all are measured against that one ruler, and

$$g(x) = \min_i \left(c_i(x) - d_i - L_f\phi_i\right) < 0$$

is the single white-box certificate that defeats any of them. The attack is unchanged from the SSA case: steer the closed loop, via an admissible inserted goal G_1', to a reachable contact that drives $g(x) < 0$ — choosing penetration depth to beat $\lambda\phi$ for the barrier filters, or just any low-authority boundary contact for the constant-η ones.